

Sweep

An SMC-based options research bot, judged by Jev

Product Blueprint | v0.1 | Prepared by Shashank | September 2026

Research and education only. This is a paper-trading and backtesting tool, not a live trading system. Nothing in this document is financial advice, and no past or simulated result implies anything about future or live-market performance.

One line: code detects Smart Money Concept structure (order blocks, fair value gaps, liquidity sweeps) and computes options mechanics (Greeks, IV rank); Jev judges confluence, context, and direction across a battery of parallel typed questions — the same split the L1vsun/JEV-Trading-BOT reference uses for order-flow, applied to a different, less rigorously validated domain.

1. Executive Summary

Sweep applies Smart Money Concept (SMC) structural analysis to options, following the architectural split proven in the L1vsun/JEV-Trading-BOT reference project: deterministic code detects patterns and computes quantitative mechanics, and Jev supplies calibrated judgment across many parallel typed questions in a single call. The reference project's plumbing — its Jev client, confidence-gated sizing mechanism, self-consistency checks, and cache/replay system — transfers with little change. Its actual quant domain (order-flow imbalance, VPIN, Kyle's lambda) does not transfer at all; SMC structure detection and options Greeks/IV computation must be built from scratch.

- **Success is measured by expected value and maximum drawdown, not hit rate.** The reference project's own results (41–57% hit rate, yet consistently beating random benchmarks) demonstrate that a sound bot does not need to be right most of the time — chasing an “accuracy” number is close to a vanity metric here.
- **SMC has no peer-reviewed validation**, unlike the order-flow-imbalance/VPIN literature the reference project draws on. It is a popular, retail-taught framework — LuxAlgo's Smart Money Concepts indicator is TradingView's single most-liked indicator — but popularity is not evidence of edge, and independent users report mixed-to-poor backtest results from SMC-based indicators in practice.
- **Two problems the reference project doesn't solve for this domain:** there is no equivalent “honest synthetic simulator” for SMC the way there is for order-flow imbalance, so validation must happen against real historical options data from day one; and sourcing a full options chain (strikes, expiries, IV) is a materially bigger, usually paid, data problem than a simple order-book CSV.

2. The Problem

Retail traders using SMC today mostly do it by eye — spotting order blocks, fair value gaps, and liquidity sweeps visually on a chart. That's slow, inconsistent between traders, and prone to hindsight bias: SMC patterns are typically defined in terms of what price did afterward (“the order block that held”), which is easy to see looking backward and genuinely ambiguous in real time. There is no widely available, code-first pipeline that detects these patterns consistently and then applies calibrated, confluence-aware judgment to an options-specific decision

(direction, strike, expiry, buy vs. sell premium) rather than gut feel.

3. Product Overview

3.1 What it does

- Detects SMC structural elements in code: swing pivots, break of structure / change of character, order blocks, fair value gaps, liquidity pools and sweeps, and the current premium/discount zone.
- Computes options mechanics in code from the underlying's options chain: Greeks (delta, theta, gamma) and IV rank/percentile for candidate strikes and expiries.
- Builds a named-bucket state description combining structural and options context — the same design principle as the reference project's state builder.
- Sends roughly 10–12 parallel typed questions to Jev per decision tick, judging confluence, structural bias, zone context, IV regime, and action — the action question asked three ways for self-consistency, exactly as in the reference project.
- Applies confidence-gated position sizing and composite scoring in code, then paper-executes and grades against a benchmark.
- Renders every run to a self-contained HTML replay dashboard, reusing the reference project's dashboard generator with added SMC-structure and options-Greeks panels.

3.2 Who uses it

- Primarily a personal research and backtesting tool, not (at this stage) a product sold to others.
- If it goes further: retail options traders who already use SMC manually and want a consistent, code-first version of their own judgment process — positioned explicitly as a research and decision-support tool, never as an autonomous money-manager.

3.3 MVP scope

Build and visually validate the SMC structure-detection code against historical charts before wiring anything to Jev — a bad structural detector just produces a confident-sounding wrong judgment on top of it. Start with one underlying and one expiry bucket (e.g. monthly options on a single liquid index or stock) to keep the options-chain data problem tractable before generalizing to more.

4. Detailed Features

4.1 SMC structure detection engine (new)

- Swing high/low pivot detection, feeding break-of-structure and change-of-character classification via pivot-sequence comparison.
- Order block identification (the last opposing candle before an impulsive move).
- Fair value gap detection (the three-candle imbalance pattern).
- Liquidity pool and sweep detection (equal highs/lows, and a swept-then-reclaimed level).
- Premium/discount zone calculation (Fibonacci position within the current swing range).
- All of the above are pure rule-based geometry in code — none of it is asked of Jev, since pattern detection over raw price series is exactly the kind of counting and multi-hop comparison Jev's own jaggedness notes flag as unreliable.

4.2 Options quant engine (new)

- Options chain ingestion: strikes, expiries, bid/ask, and quoted or computed implied volatility.
- Greeks computation (delta, theta, gamma) per candidate strike using a standard options-pricing library.
- IV rank / IV percentile relative to the underlying's own recent history.
- Days-to-expiry bucketing and a shortlist of candidate strikes/expiries for the decision layer to consider.

4.3 State builder (adapted from the reference project)

- Combines structural and options context into named buckets Jev reads well — e.g. “bullish order block at [level], price returned to it and held,” “liquidity swept below the prior swing low, then reclaimed,” “IV rank: elevated.”
- Only the fields the current question set needs are included, following the reference project's own state-minimization discipline.

4.4 Jev decision layer (mechanism reused, questions rebuilt)

A code-side materiality filter sits in front of this layer: a symbol is only sent to Jev when its structural or options state has moved meaningfully since it was last judged, keeping call volume (and rate-limit exposure) proportional to what's actually changing, not to how many symbols are tracked.

Question	Type	What it judges
structure_bias	Choice	Bullish / bearish / ranging, given detected BOS/CHoCH
liquidity_event	Noul	Genuine sweep-and-reverse vs. continuation
ob_confluence	Score	How much confluence supports the detected order block
fvg_fill_expectation	Choice	Likely filled vs. likely respected as support/resistance
zone_context	Choice	Premium / discount / equilibrium
iv_regime	Choice	Elevated / normal / low — informs buying vs. selling premium

Question	Type	What it judges
setup_quality	Score	Aggregate confluence across all signals above
action × 3	Choice	Buy calls / buy puts / sell premium / no trade — asked three ways for self-consistency, exactly as in the reference project
exit_now	Noul	Speculative fan-out, asked every tick, only read once in a position

4.5 Validation and backtesting harness (new, the hardest part)

- No honest synthetic simulator is available for SMC the way one exists for order-flow imbalance — there is no agreed generative model of “what an order block actually is” to build a clean test against, so validation must happen against real historical options data from the start.
- Walk-forward, strictly out-of-sample testing; expected value and maximum drawdown are the headline metrics, not hit rate.
- A dedicated look-ahead-bias test suite, extended from the reference project's own, with extra attention to SMC-specific hindsight bias: a pattern like “the order block that held” is partly defined by what price did afterward, so detection code must be strictly causal — using only information available at the moment of the decision, never a full window that includes the outcome.

4.6 Replay dashboard (shell reused, panels extended)

- Reuses the reference project's self-contained HTML replay generator.
- Adds an SMC structure overlay panel (pivots, order blocks, FVGs, liquidity levels) and an options Greeks/IV panel alongside the existing price, state, and decision panels.

4.7 Plumbing (reused with little change)

- The Jev client itself: authentication, retries on 429/529, model pinning and aliases.
- Confidence-gated sizing mechanism and composite scoring in code (thresholds and weights re-tuned for this domain, but the mechanism is unchanged).
- Self-consistency checks and speculative fan-out.
- The cache/replay system, letting decision logic be iterated on without repeated API or data costs.

5. System Architecture

Price/candle data + options chain data → SMC structure detector (code) and options quant engine (code), run in parallel → state builder (named buckets) → Jev (10–12 parallel typed questions) → decision (composite score, confidence-gated sizing, self-consistency check) → paper execution and grading against a benchmark → replay dashboard and decision log.

The one architectural addition over the reference project is that two independent detection/computation paths (structure and options mechanics) must both complete and feed the same state builder before a Jev call is made — the reference project only had one quant path (order-flow) to synchronize.

5.1 Performance, kept deliberately low-risk

A faster live loop and scanning several underlyings at once were both explicit goals here, but a streaming market-data feed and a compiled hot path (Rust or numba) were considered and deliberately deferred — either would mean two separate implementations of the same SMC detection logic (one for backtesting, one for live), which can silently drift apart over time and quietly invalidate what the backtest was supposed to prove. The stack below stays a single, plain-Python implementation shared by both the backtester and the live loop, with five changes that meaningfully tighten the loop without introducing that risk:

- **Proper vectorization.** Structure detection and Greeks computation operate on whole numpy/pandas arrays, never a Python for loop per contract or per candle — often a 10–50x speedup on its own, with no new moving parts.
- **Batched data fetches.** Pull multiple underlyings in a single request where the provider supports it, instead of one request per symbol per tick.
- **Concurrent fetches via asyncio.** Where batching isn't available, fire requests for every tracked underlying at once rather than looping through them sequentially — ordinary async I/O, not a new architecture.
- **Recompute only on change.** Skip a full recompute every tick; re-run Greeks only when the underlying has moved past a small threshold, re-check structure only when a new candle has actually closed.
- **A Jev materiality filter.** Only enqueue a Jev call for a symbol when its structural or options state has moved enough since it was last judged — this is the single biggest lever against both cost and rate limits as the tracked-symbol count grows.

***Honest ceiling:** this realistically tightens the loop from 10s to roughly 3–5s, and comfortably supports low double digits of tracked underlyings at once, all inside one auditable codebase. It will not reach a true 1s tick across hundreds of symbols — that genuinely requires the streaming feed and compiled path this design trades away, and that trade was made deliberately.*

6. Technology Stack

Layer	Choice	Why
Core language / structure	Python, reusing the reference project's package layout — one implementation shared by both backtester and live loop	Keeps the reused plumbing (Jev client, cache/replay, CLI) drop-in compatible, and avoids a second implementation of the same detection logic drifting out of sync with the first.
SMC structure detection	Vectorized pandas / numpy, rule-based	This is geometry and sequence comparison, not a machine-learning problem — keep it deterministic and inspectable, and written as whole-array operations rather than per-candle Python loops.
Options pricing / Greeks	py_vollib or QuantLib-Python, vectorized across a chain	Standard, well-tested options-pricing libraries, called on whole arrays of contracts rather than one at a time in a loop.
Options chain data	A paid provider (e.g. Tradier, Massive/formerly Polygon.io) with batched multi-symbol requests	Batching cuts network round trips directly; budget for this as a real cost, not an afterthought.
Concurrency	asyncio for concurrent per-underlying fetches	Ordinary async I/O — fires requests for every tracked underlying at once instead of looping sequentially, with no new architecture or compiled dependency involved.
Jev access	Direct TypeSafe client, reused from the reference project's typesafe.py	No reason to rebuild a working, tested integration.

Layer	Choice	Why
Backtesting engine	A custom event-driven backtester, not a vectorized one	Options payoffs are path-dependent (assignment, early exercise, theta decay along the path) in a way a simple vectorized backtest can silently get wrong.
Dashboard	The reference project's replay.html generator, extended with new panels	Reuse a working, self-contained output format rather than building a new dashboard stack.
Storage / cache	JSONL run cache, matching the reference project's --cache convention	Keeps the reused cache/replay tooling compatible without translation.
CI	GitHub Actions running pytest, including a dedicated look-ahead-bias suite	Directly extends the reference project's own testing discipline rather than inventing a new one.

7. Build Plan

Phase	Focus	Deliverable
0. Scope	Setup	Pick one underlying and one options-chain data source; confirm budget for paid chain data before writing any detection code.
1. Reuse plumbing	Weeks 1–2	Fork the reference project's Jev client, cache/replay system, and CLI scaffolding; strip out its order-flow-specific quant code entirely.
2. Structure detection	Weeks 2–4	Build the SMC structure-detection engine; validate it visually against historical charts — no Jev calls yet at this stage.
3. Options quant engine	Weeks 4–5	Wire in chain data, compute Greeks and IV rank against real historical options data for the chosen underlying.
4. Jev integration	Weeks 5–6	Build the new state builder and question set; wire to Jev; build the confidence-gated sizing and composite-scoring logic.
5. Validation harness	Weeks 6–8	Walk-forward, strictly out-of-sample backtesting against real historical data; extend the look-ahead-bias test suite for SMC-specific hindsight risk; report expected value and drawdown, not hit rate.
6. Dashboard	Weeks 8–9	Extend the reused replay dashboard with SMC-structure and options-Greeks panels.
7. Performance pass	Week 9	Vectorize any remaining per-contract or per-candle loops; batch and parallelize (asyncio) data fetches; add the recompute-only-on-change triggers and the Jev materiality filter — all within the same single-implementation codebase.
8. Extended paper trading	Weeks 10+	Run in real time, paper only, for an extended period before any consideration of anything beyond paper trading.

8. Reference Points (Not Direct Competitors)

None of these are quite the same product — useful as reference points for what already exists in adjacent pieces of this space.

Project	URL	What it actually is
L1vsun/JEV-Trading-BOT	github.com/L1vsun/JEV-Trading-BOT	The direct architectural reference: code computes order-flow quant features, Jev judges 12 parallel typed questions. Spot/futures, not options; academic microstructure, not SMC.
LuxAlgo	luxalgo.com	Retail indicator suite on TradingView; its Smart Money Concepts script is the platform's single most-liked indicator. Fires visual signals on a chart — no options integration, no calibrated confidence layer, and independent users report mixed backtest results.
TrendSpider	trendspider.com	Standalone charting platform with automated technical analysis, a no-code Strategy Tester, and options tooling in the same workspace — broad but no-code, not a programmable SMC-plus-options judgment pipeline.

Project	URL	What it actually is
QuantConnect	quantconnect.com	Open, code-first backtesting and live-deployment platform (Python or C#) with genuine options support — Greeks, IV, multi-leg strategies, broker integrations. No SMC-specific detection and no Jev-style calibrated judgment layer.

9. What Would Make This Different, Not Just Another SMC Indicator

#	Difference	What exists today	What Sweep does instead
1	SMC and options combined	LuxAlgo does SMC pattern detection on price charts only; QuantConnect does options backtesting with no SMC detection; TrendSpider bundles pieces of both but stays no-code.	One pipeline detects SMC structure and computes options mechanics together, feeding a single decision.
2	A judgment layer, not just a signal	LuxAlgo and TrendSpider fire a signal the moment a pattern is detected — pattern present equals signal.	Jev adds a calibrated, confluence-aware judgment on top of detected patterns, gated by confidence rather than firing on pattern presence alone.
3	Honest validation posture by design	Independent users report SMC-indicator backtests underperforming simple baselines in practice, and even vendors caution against treating AI-assisted backtests as proof of live performance.	Headline metrics are expected value and drawdown, not hit rate, with real historical validation required before any live consideration — built to resist the trap those other tools fall into.
4	Self-consistency and confidence gating	None of the three reference points ask the same directional question multiple ways and require agreement before acting.	Inherited directly from the reference project: the action question is asked three ways, and sizing is gated on confidence, not a single yes/no signal.
5	A fully local, replayable research loop	QuantConnect requires cloud compute and data subscriptions to iterate on a strategy.	The cache/replay system inherited from the reference project lets decision logic be refined without repeated data or API costs.

10. Risks & Open Questions

Risk	Why it matters	Mitigation
SMC's unproven status	Popularity (TradingView's #1 indicator) is not evidence of edge; independent users report backtests underperforming simple baselines.	Treat SMC as an unvalidated hypothesis being tested, not an established edge being implemented — design the validation harness to be capable of concluding “this doesn't work.”
Options-specific risk profile	Leverage, time decay, and expiry/assignment mechanics mean a single wrong high-conviction trade can lose the full premium fast — a materially different risk shape than the reference project's spot/futures approach.	Paper trade only, for an extended period, before any other consideration; track drawdown as closely as expected value.
No honest synthetic simulator	Unlike order-flow imbalance, there's no agreed generative model of SMC structure to build a clean synthetic test against.	Validate against real historical options data from day one; budget more time for validation than the reference project needed.

Risk	Why it matters	Mitigation
Options chain data cost and access	Full chain history (strikes, expiries, IV) is a materially bigger, usually paid, data problem than a simple order-book CSV.	Budget for a paid data provider or broker sandbox API from the start rather than assuming free data will be sufficient.
Hindsight bias in structure detection specifically	SMC patterns like an order block are partly defined by what price did afterward, making look-ahead bias an easier trap here than in continuous order-flow metrics.	Extend the look-ahead-bias test suite with SMC-specific causality checks — detection code must use only information available at the moment of the decision.

This blueprint reflects planning discussion as of September 2026. It is a research and education draft, built on the publicly documented design of the L1vsun/JEV-Trading-BOT project (credit that prior art clearly in any public release). Nothing in this document is financial advice; nothing here should be read as a claim that SMC-based options trading is profitable.